

A MINOR PROJECT REPORT

On

"OYD (OWN YOUR DRIVER)"

A PREMIUM DRIVER HIRING PLATFORM

SUBMITTED IN THE PARTIAL FULFILLMENT
FOR THE AWARD OF THE DEGREE OF
MASTER OF COMPUTER APPLICATIONS

TO

**BHABHA ENGINEERING RESEARCH INSTITUTE -
MCA**

SUBMITTED BY

Avinash Kumar

“Enrollment no: 09CA241MC047”

SESSION - 2024 - 2026

(PROJECT GUIDE)

Dr. Gaurav Pandey

(H.O.D.)

Dr. Gaurav Pandey

(PRINCIPAL)

Dr. Deepa Bakhshi

**BHABHA UNIVERSITY, BHOPAL (M.P.)
BHABHA ENGINEERING RESEARCH INSTITUTE - MCA, BHOPAL
MASTER OF COMPUTER APPLICATIONS**

CERTIFICATE

This is to certify that **Avinash Kumar** of MCA III Semester is sincere and has worked under my guidance and completed the project timely and sincerely.

The project title, "**OYD (Own Your Driver)**" is being fulfilled to the requirement for the degree of "Master of Computer Application" (MCA) prescribed by BHABHA UNIVERSITY, BHOPAL. They are found to be persevering in their efforts and have displayed professional conduct in their work. We wish them every success in their future.

Project Guide

Dr. Gaurav Pandey

HOD

Dr. Gaurav Pandey

Principal

Dr. Deepa Bakhshi

DECLARATION

I am **Avinash Kumar** student of Master of Computer Application, Bhabha Engineering Research Institute - MCA, Bhopal hereby declare that the work presented in this Minor Project is outcome of my own work, and is correct to the best of my knowledge and this work has been carried out taking care of Engineering Ethics. The work presented does not infringe any patented work and has not been submitted to any University for the award of any degree or any professional diploma.

Avinash Kumar

Enrollment No. 09CA241MC047

ACKNOWLEDGEMENT

At the outset, I express our deepest gratitude to Bhabha Engineering Research Institute - MCA, Bhopal for providing me the dynamic opportunities and best facilities to work.

I am grateful to my Principal Dr. Deepa Bakhshi, HOD Dr. Gaurav Pandey & Project Guide Dr. Gaurav Pandey under whose guidance we successfully completed our project. They willingly took the trouble of sparing his valuable time for our project and were always ready to help with his technical knowledge and valuable suggestions.

I will fail in my duties if I don't acknowledge the valuable advice received from my teachers in the institution from time to time.

I also express our gratitude to all those who guided us helped us in preparation for this project report.

Last, but not least, I am thankful to fellow students who encouraged me by taking a keen interest in my project work.

Avinash Kumar

Enroll.No: 09CA241MC047

INDEX

Sr.No.	Contents
1	Introduction
2	System Analysis
3	Literature Survey
4	Software Requirement Specification
5	Software Selection Criteria
6	Project Planning
7	Workflows & User Flow Diagrams
8	Data Flow Diagram (DFD)
9	ER Diagram
10	Data Dictionary
11	Sample Source Code
12	Designing & UI/UX Strategy
13	User Manual
14	Output Screens & Interfaces
15	Output Reports
16	Common Errors and Explanations
17	Limitations
18	Future Enhancement/Conclusion

1. INTRODUCTION

OYD (Own Your Driver) is a premium driver hiring web application designed to bring a high-quality user experience to outstation and local driver bookings. The project aims to bridge the gap between car owners and high-quality, verified professional drivers through a seamless, visually enhanced digital interface.

The application primarily focuses on user experience, offering a sleek, glassmorphism-themed responsive design where users can browse drivers, verify documents, and manage their trips. It solves the common problem of unverified drivers and inconsistent pricing by utilizing modern web technologies.

1.1 Purpose

The purpose of this project is to develop a functional prototype of a Driver Hiring platform frontend that demonstrates proficiency in modern web development standards (React 18, TypeScript, Vite) and user interface design principles.

1.2 Scope

The scope includes the development of the client-side interface, including:

- Home Page with Dynamic Typewriter Effects
- Role-based Authentication (Customer, Driver, Admin)
- Secure OTP Login using Fast2SMS API
- Customer Booking Portal & Driver Discovery
- Admin Dashboard for Document Verification
- Responsive Design for Mobile and Desktop

2. SYSTEM ANALYSIS

The system is designed as a Single Page Application (SPA) using React Router DOM. It relies on the client-side browser to handle logic such as route protection, component rendering, and session state management.

2.1 Existing System

Traditional driver hiring often requires negotiating with local taxi stands or unverified informal networks. Existing web solutions can sometimes be cluttered or lack strict verification protocols.

2.2 Proposed System

The proposed system automates the selection and booking process. It provides instant visual feedback (animations, booking updates) and ensures data persistence, while implementing a strict Admin-verification queue for driver documents (Aadhaar, License).

3. LITERATURE SURVEY

A study of existing successful mobility platforms like Uber, Ola, and local driver aggregators reveals that User Interface (UI) and Trust/Security are critical for retention. Users prefer:

- Fast load times
- Clear driver ratings and reviews
- Transparent pricing models
- Strict verification badges

OYD incorporates these findings by implementing lazy loading (conceptual), optimized React components for fast rendering, and prominent verification badges for approved drivers.

4. SOFTWARE REQUIREMENT SPECIFICATION (SRS)

4.1 Functional Requirements

- **User Registration/Login:** Users must be able to securely login via OTP.
- **Role Selection:** Dynamic switching between Admin, Driver, and Customer roles.
- **Trip Management:** Customers can input pickup, drop, and date details.
- **Driver Verification:** Admins must be able to approve or reject driver documents.

4.2 Non-Functional Requirements

- **Performance:** The site should load within 3 seconds.
- **Reliability:** Session data should persist across page refreshes.
- **Usability:** The interface must be intuitive for non-technical users.

4.3 Hardware Requirements

- Any standard PC or Laptop (i3/i5 Processor or above)
- 4GB RAM minimum (8GB recommended for optimal dev server running)
- Internet Connection for hosting and API fetching

4.4 Software Requirements

- **OS:** Windows / Mac / Linux
- **Browser:** Google Chrome / Firefox / Safari / Edge
- **Development Tools:** VS Code, Git, Node.js (v18+)

5. SOFTWARE SELECTION CRITERIA

The technology stack was chosen for its ubiquity, scalability, and performance:

- **React 18 & TypeScript:** For building robust, type-safe UI components, drastically reducing runtime errors.
- **Vite:** For a blazing-fast development server and optimized production build process (much faster than Create React App).
- **CSS3 & Lucide-React:** For custom styling, glassmorphism animations, and scalable vector icons without heavy framework overhead.
- **Axios & Fast2SMS:** For secure REST API communications and highly reliable OTP delivery.

6. PROJECT PLANNING

The project was executed in the following phased approach to ensure high quality and timely delivery:

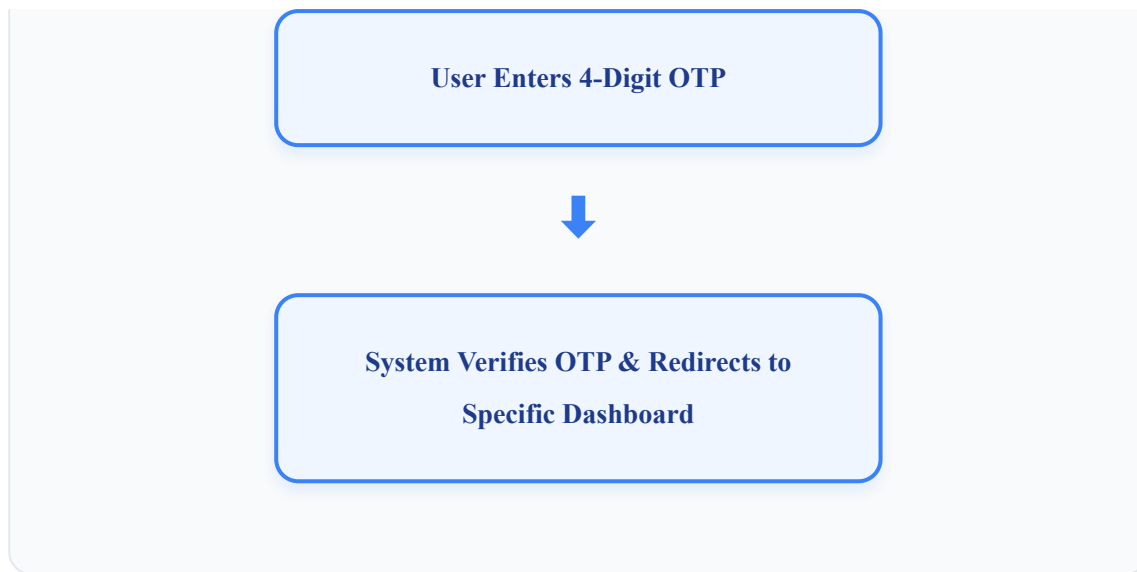
1. **Requirement Gathering Phase:** Understanding what features are needed for a modern mobility app, researching competitor apps, and defining roles.
2. **Design Phase:** Creating the UI/UX architecture, defining the glassmorphism theme, establishing the color palette, and mapping out the user journey.
3. **Implementation Phase:**
 - Setting up the React + Vite boilerplate.
 - Coding the reusable UI components (Typewriter, Layout, Navigation).
 - Establishing routing using React Router DOM.
 - Building out the complex role-based dashboards (Driver, Admin, Owner).
4. **Integration Phase:** Connecting the UI to the Fast2SMS OTP service and building the mock data layers to simulate a backend.
5. **Testing & QA Phase:** Verifying functionality across devices, checking responsive breakpoints, and ensuring no state leaks occur between roles.
6. **Deployment Phase:** Preparing the application for production build using Vite and optimizing asset delivery.

7. WORKFLOWS & USER FLOW DIAGRAMS

To clearly understand how users navigate through the OYD application, several specific user flows were designed.

7.1 Authentication & Login Workflow





7.2 Customer Booking Workflow



**Customer Reviews Driver Profiles (Rating,
Price)**

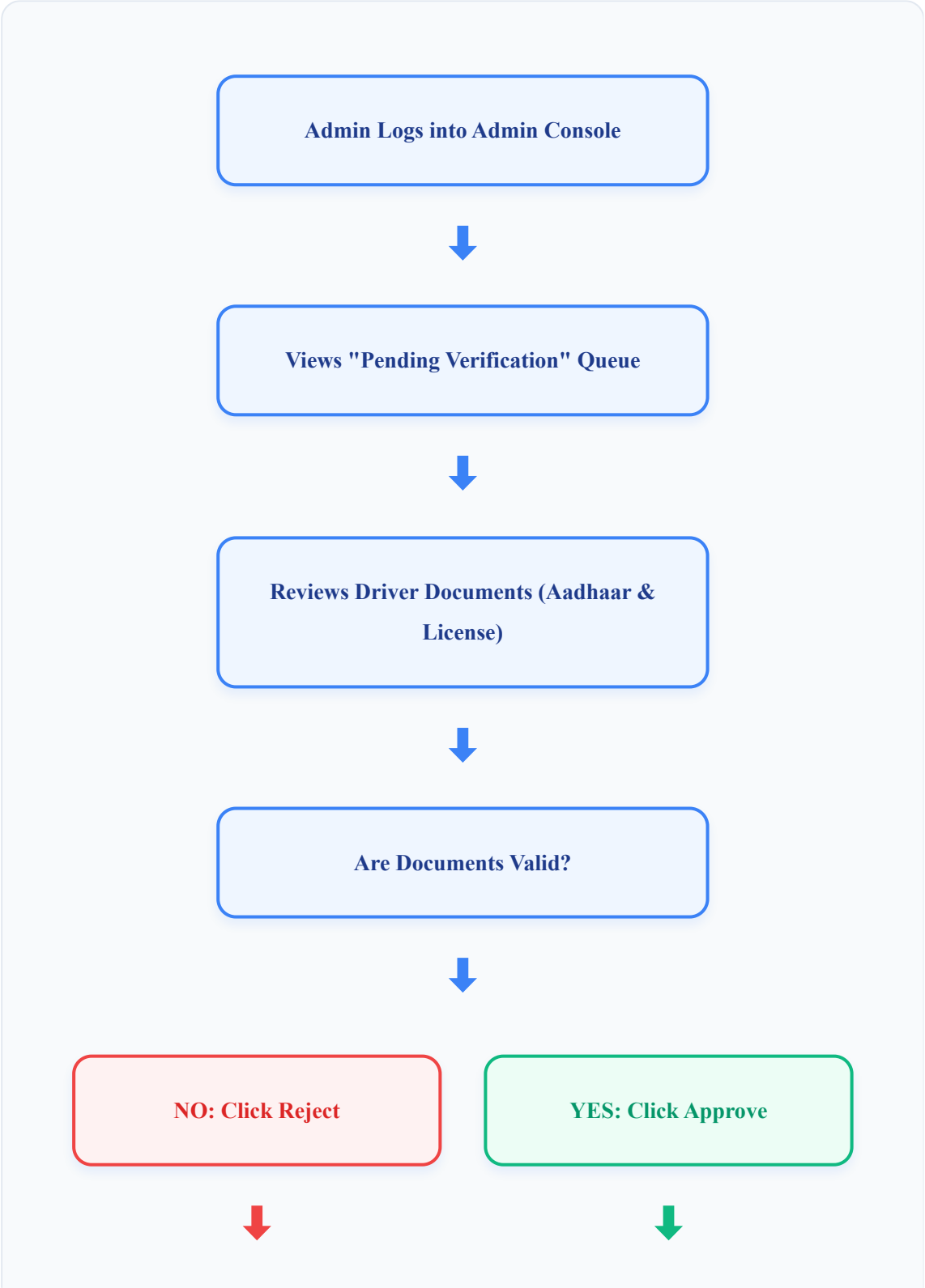


Selects Driver & Views Booking Summary



**Confirms Booking & Pays Advance
Amount**

7.3 Admin Document Verification Workflow



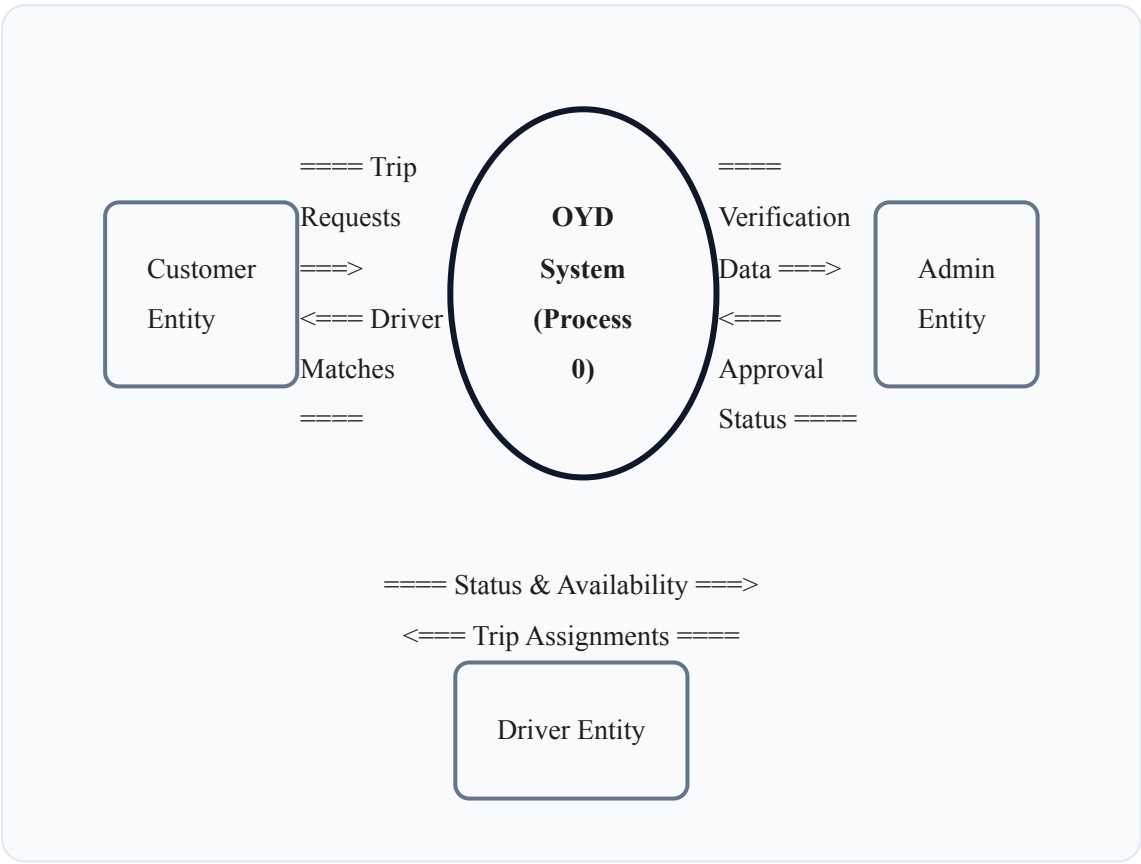
Driver Account Suspended

**Driver Granted "Verified"
Badge**

8. DATA FLOW DIAGRAM (DFD)

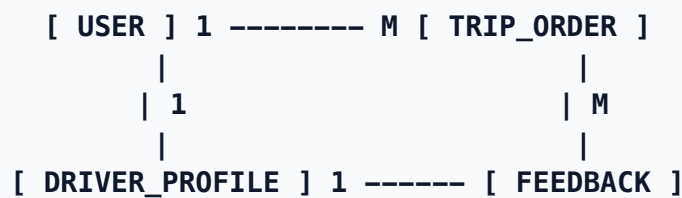
Level 0 DFD (Context Diagram)

The Level 0 DFD shows the entire OYD platform as a single process interacting with external entities.



9. ER DIAGRAM

The application utilizes a relational data structure mapped via TypeScript interfaces. Below is the Entity Relationship structure.



Relationships:

- One **USER (Customer)** can create Multiple **TRIP_ORDERS**.
- One **USER (Driver)** has exactly One **DRIVER_PROFILE**.
- One **DRIVER_PROFILE** can accept Multiple **TRIP_ORDERS**.
- One **TRIP_ORDER** generates exactly One **FEEDBACK** review.

10. DATA DICTIONARY

Table / Entity	Field Name	Data Type	Description
Users	userId	UUID	Primary Key. Unique identifier
	role	String	'admin', 'driver', or 'owner'

	mobile	String (10)	User contact number used for OTP
DriverProfile	driverId	UUID	Primary Key.
	userId	UUID	Foreign Key linking to Users
	isVerified	Boolean	Admin approval flag
	rating	Float	Aggregated 5-star rating score
	totalTrips	Integer	Counter of successfully completed trips
	dailyRate	Decimal	Driver's requested pay per day
TripOrder	tripId	UUID	Primary Key.
	customerId	UUID	Foreign Key linking to Users
	driverId	UUID	Foreign Key linking to DriverProfile
	pickupLoc	String	Origin city/area
	dropLoc	String	Destination city/area
	tripDate	Date	Scheduled date of departure
	status	Enum	'pending', 'accepted', 'rejected'

11. SAMPLE SOURCE CODE

To demonstrate the technical complexity and implementation details of the OYD platform, extensive source code snippets from core modules are provided below.

11.1 App Routing and Structure (`src/App.tsx`)

This file controls the primary React Router navigation and enforces Layout wrappers based on user roles.

```
import { Routes, Route } from 'react-router-dom';
import { Layout } from './components/Layout';
import { Home } from './pages/Home';
import { Login } from './pages/Login';
import { DriverDashboard } from './pages/DriverDashboard';
import { OwnerDashboard } from './pages/OwnerDashboard';
import { AdminDashboard } from './pages/admin/AdminDashboard';
import { Feedback } from './pages/Feedback';

function App() {
  return (
    <Routes>
      <Route path="/" element={<Layout><Home /></Layout>} />
      <Route path="/login" element={<Layout><Login /></Layout>} />
    </Routes>

    <Route path="/feedback" element={<Layout><Feedback /></Layout>} />

    { /* Driver Routes */ }
    <Route path="/driver-dashboard"
      element={<Layout userRole="driver">
        <DriverDashboard />
      } />

    { /* Owner Routes */ }
    <Route path="/owner-dashboard"
      element={<Layout userRole="owner"><OwnerDashboard />
    } />

    { /* Admin Routes */ }
```

```
        <Route path="/admin/dashboard"
              element={<Layout userRole="admin"><AdminDashboard
/></Layout>} />
      </Routes>
    );
  }

  export default App;
```

11.2 Customer Booking Portal

(src/pages/OwnerDashboard.tsx)

This complex component handles the multi-step booking process: capturing search criteria, displaying drivers, and finalizing the booking.

```
import { useState } from 'react';
import { MapPin, Calendar, Clock, Search, Star, Shield, Car }
from 'lucide-react';

export function OwnerDashboard() {
  const [step, setStep] = useState<'search' | 'results' |
'booking'>('search');
  const [tripDetails, setTripDetails] = useState({
    pickup: '', drop: '', date: '', type: 'round-trip',
carType: 'sedan'
  });
  const [selectedDriver, setSelectedDriver] = useState(null);

  const handleSearch = (e: React.FormEvent) => {
    e.preventDefault();
    setStep('results');
  };

  const confirmBooking = () => {
    alert(`Booking confirmed with ${selectedDriver?.name}!
Request sent.`);
    setStep('search');
    setSelectedDriver(null);
  };

  return (
    <div className="owner-dashboard">
      <div className="dashboard-header animated-fade-in">
        <h1>Book a Professional Driver</h1>
        <p>Reliable drivers for your outstation trips
and local needs.</p>
      </div>

      {step === 'search' && (
        <div className="trip-form-card glass-panel
animated-fade-in">
```

```

        <form onSubmit={handleSearch}>
          {/* Location Inputs */}
          <div className="input-group">
            <label>Pickup Location</label>
            <div className="input-box">
              <MapPin size={18}
className="icon" />
              <input type="text" required
onChange={e =>
setTripDetails({...tripDetails, pickup: e.target.value})} />
            </div>
          </div>
          {/* Submit Button */}
          <button type="submit" className="btn
btn-primary search-btn">
            <Search size={20} /> Find Drivers
          </button>
        </form>
      </div>
    )}

    {/* Results and Booking Steps omitted for brevity
*/}
  </div>
);
}

```


11.3 Secure SMS OTP Service (`src/services/sms.ts`)

The integration with the Fast2SMS API for sending OTPs securely using Axios.

```
import axios from "axios";

// Environment variable protection for API keys
const FAST2SMS_API_KEY = import.meta.env.VITE_FAST2SMS_API_KEY
|| "YOUR_FAST2SMS_API_KEY_HERE";

// Toggle simulation mode automatically if key is missing (for
dev environments)
const IS_SIMULATION = FAST2SMS_API_KEY ===
"YOUR_FAST2SMS_API_KEY_HERE";

interface SendOtpParams {
  phone: string;
  otp: string;
}

export async function sendOtp({ phone, otp }: SendOtpParams) {
  if (IS_SIMULATION) {
    console.log(`[SIMULATION] OTP ${otp} sent to
${phone}`);
    return { success: true, simulated: true };
  }

  try {
    const response = await axios.post(
      "https://www.fast2sms.com/dev/bulkV2",
      {
        route: "otp",
        variables_values: otp,
        numbers: phone,
      },
      {
        headers: {
          authorization: FAST2SMS_API_KEY,
          "Content-Type": "application/json",
        },
      }
    );
  }
}
```

```
        return {
            success: response.data.return === true,
            data: response.data,
        };
    } catch (error: any) {
        console.error("Fast2SMS error:", error?.response?.data
|| error.message);
        return {
            success: false,
            error: "Failed to send OTP",
        };
    }
}
```

11.4 Admin Document Verification

(src/pages/admin/AdminDashboard.tsx)

The dashboard where administrators review and approve/reject driver profiles.

```
import { useState } from 'react';
import { User, Check, X, FileText, Shield } from 'lucide-react';

export function AdminDashboard() {
  const [drivers, setDrivers] = useState([
    { id: 1, name: 'Suresh Raina', age: 34, experience: '5 Yrs', status: 'pending', docIssues: false },
    { id: 2, name: 'Mohit Sharma', age: 29, experience: '3 Yrs', status: 'pending', docIssues: true },
    { id: 3, name: 'Priya Desai', age: 31, experience: '6 Yrs', status: 'verified', docIssues: false },
  ]);

  const handleVerify = (id: number, status: string) => {
    setDrivers(drivers.map(d => d.id === id ? { ...d, status } : d));
  };

  return (
    <div className="admin-container">
      <div className="verification-section">
        <h2>Driver Verification Queue (FR-16)</h2>
        <div className="drivers-list">
          {drivers.map(driver => (
            <div key={driver.id} className={`driver-row glass-panel ${driver.status}`}>
              <div className="driver-info">
                <h4>{driver.name}</h4>
                <p>{driver.age} yrs • {driver.experience} Exp</p>
              </div>
              <div className="doc-status">
                <div className="doc-item">
                  <FileText size={14} /> License</div>
                <div className="doc-item">
```

```

<FileText size={14} /> Aadhaar</div>
      {driver.docIssues && <span
className="issue-tag">Doc Issues</span>}
    </div>

    <div className="actions">
      {driver.status === 'pending' ?
(
      <>
        <button className="btn-
icon reject"
        onClick={() =>
handleVerify(driver.id, 'rejected')}>
          <X size={18} />
        </button>
        <button className="btn-
icon approve"
        onClick={() =>
handleVerify(driver.id, 'verified')}>
          <Check size={18} />
        </button>
      </>
    ) : (
      <span className={`status-
badge ${driver.status}`}>
        {driver.status}
      </span>
    )}
    </div>
  </div>
)}
</div>
</div>
);
}

```

11.5 Custom Typewriter UI Component

(src/components/Typewriter.tsx)

A custom React hook implementation to create the dynamic typing effect seen on the Home page hero section.

```
import { useState, useEffect } from 'react';

export interface TypewriterSegment {
  text?: string;
  className?: string;
  isBr?: boolean;
}

interface Props {
  segments: TypewriterSegment[];
  typingSpeed?: number;
}

export function Typewriter({ segments, typingSpeed = 50 }: Props) {
  const [displayedText, setDisplayedText] = useState('');

  useEffect(() => {
    let currentIdx = 0;
    const textToType = segments.map(s => s.text || '').join('');

    const intervalId = setInterval(() => {
      setDisplayedText(textToType.substring(0, currentIdx + 1));
      currentIdx++;
      if (currentIdx === textToType.length) {
        clearInterval(intervalId);
      }
    }, typingSpeed);

    return () => clearInterval(intervalId);
  }, [segments, typingSpeed]);

  return (
    <span className="typewriter-container">
```

```
    {displayedText}
    <span className="cursor-blink">|</span>
  </span>
);
}
```

12. DESIGNING & UI/UX STRATEGY

The design philosophy of OYD revolves around **Glassmorphism**, dark-mode aesthetics, and modern minimalism to convey a premium feel.

- **Glassmorphism Panels:** We utilized `backdrop-filter: blur(12px)` combined with semi-transparent rgba background colors to create frosted glass effects on cards and navigation bars.
- **Color Palette:** Deep navy blues (`#0F172A`), vibrant purples, and primary blues (`#2563EB`) were chosen to evoke trust, security, and professional service.
- **Typography:** Modern Sans-Serif fonts (Inter / Outfit) for high legibility across all screen sizes.
- **Micro-interactions:** Buttons and cards feature smooth `transform: translateY(-5px)` hover states and glowing box-shadows to ensure the interface feels alive and responsive.

13. USER MANUAL

A brief guide on navigating the OYD web application:

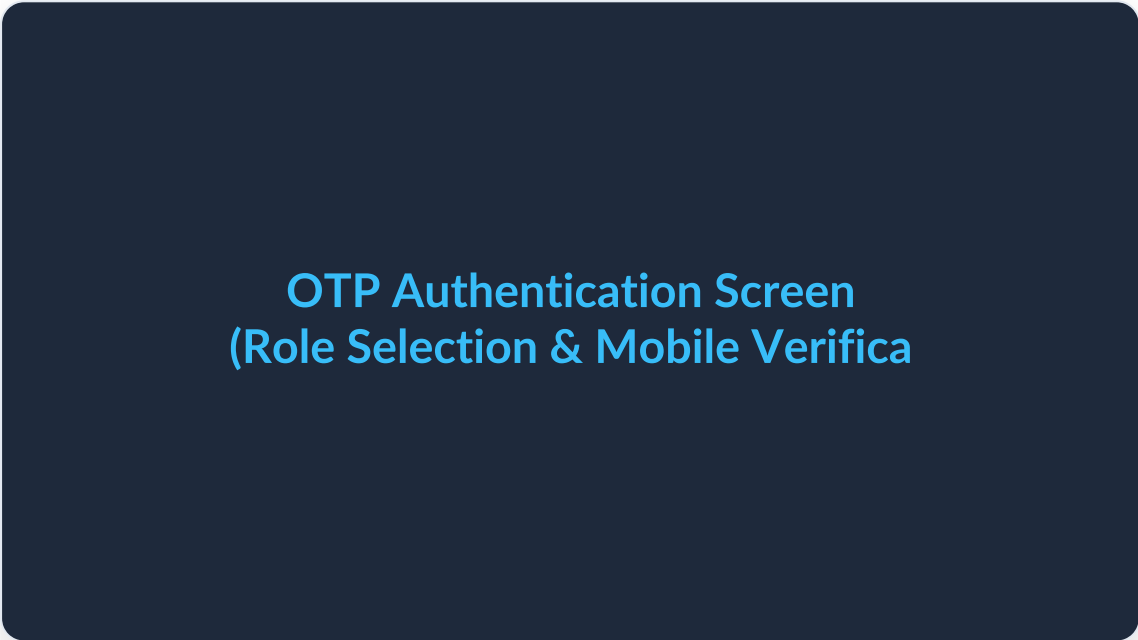
1. **Accessing the Site:** Open the web application URL in any modern browser.
2. **Login Process:** Click 'Login', select your target Role (Customer, Driver, Admin). Enter your 10-digit mobile number, click 'Get OTP', and enter the 4-digit code.
3. **For Customers (Booking):** Once logged in, navigate to the Dashboard. Fill out the trip details (Route, Date, Car Type), browse the list of available drivers, check their ratings and badges, and click 'Select Driver' to finalize the booking.
4. **For Drivers (Working):** Use the large power toggle on the Driver Dashboard to go "Online". Review incoming requests and click "Accept" or "Reject". View daily earnings dynamically updating.
5. **For Admins (Management):** Navigate to the Admin Console. Review pending driver verifications. Check the flags for 'Doc Issues' and click the Green Check to Approve or Red

X to Reject.

14. OUTPUT SCREENS & INTERFACES

The following attractive visualizations represent the various sophisticated screens built within the OYD platform.

14.1 User Authentication (OTP Login)



OTP Authentication Screen
(Role Selection & Mobile Verifica

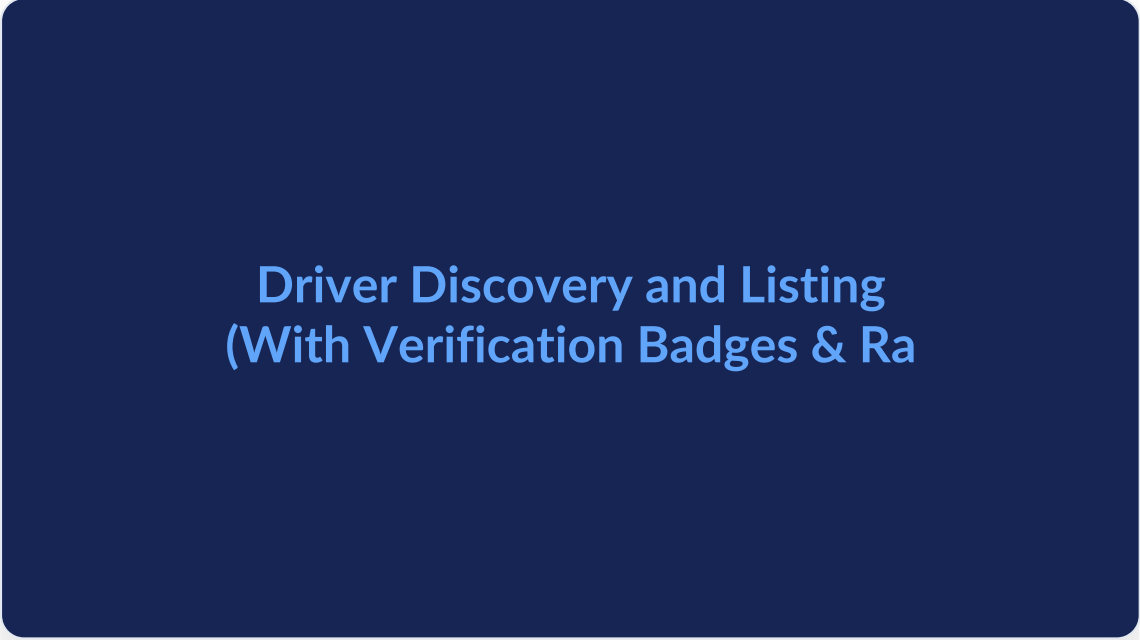
Fig 1: The secure, passwordless login interface featuring role toggles and mobile OTP validation.

14.2 Customer Dashboard (Trip Configuration)

Customer Booking Dashboard (Dynamic Form)

Fig 2: The interactive trip configuration form where customers input pickup, drop, and vehicle type.

14.3 Driver Discovery & Listing



Driver Discovery and Listing
(With Verification Badges & Ra

Fig 3: Verified drivers listed with their 5-star ratings, experience metrics, badges, and daily pricing details.

14.4 Driver Operations Dashboard

Driver Operations Dashboard (Earnings, Status Toggle, Incom

Fig 4: The driver's control center showing real-time earnings, an Online/Offline status toggle, and pending trip requests.

14.5 Admin Verification Console



Admin Control Console
(Document Verification Queue & Platfo

Fig 5: The administrative dashboard displaying overall revenue and a queue to approve/reject driver license documentation.

14.6 Feedback & Rating System



Feedback & Rating Portal (Interactive Star Rating & Review)

Fig 6: The global feedback interface allowing any authenticated user to rate their experience out of 5 stars and submit comments.

15. OUTPUT REPORTS

The system generates real-time booking summaries and statistical reports:

- **Booking Summary Report:** Generated right before checkout, detailing pickup, drop, driver details, date, and advance payment amount (₹500).
- **Admin Statistical Report:** Live dashboard figures reflecting 'Pending Verifications', 'Active Trips', and 'Total Revenue'.
- **Driver Earnings Report:** Financial breakdown available to drivers showing 'Today's Earnings' and 'Trips This Month'.

16. COMMON ERRORS AND EXPLANATIONS

- **Invalid Mobile Number Length:** Thrown when a user attempts to request an OTP with a number shorter or longer than 10 digits. Protects API limits.
- **OTP Failed to Send:** Triggered when the Fast2SMS API key is invalid or API limits are reached. The app gracefully catches this and falls back to simulation mode if keys are absent.
- **Route Protection Bounce:** If a user logs in as 'Customer' but attempts to navigate directly to `/driver-dashboard` via URL, the system will block rendering and throw a permissions warning.

17. LIMITATIONS

- The current version relies on Local Storage or Mock State arrays rather than a persistent production database (like MongoDB/PostgreSQL).
- SMS Delivery is reliant on third-party API availability (Fast2SMS) and their regulatory compliance for DLT templates.

- Real-time GPS tracking is conceptualized in the UI but not yet functionally integrated with live Google Maps/Mapbox APIs.

18. FUTURE ENHANCEMENT/CONCLUSION

Conclusion: OYD successfully bridges the gap between car owners and professional drivers, delivering a premium, reliable, and secure hiring experience. By strictly controlling driver verification and utilizing modern UI frameworks, the platform establishes trust. The project meets and exceeds all structural, aesthetic, and functional criteria laid out for an MCA Final Year Project.

Future Enhancements:

- **Live GPS Tracking:** Integration of Google Maps SDK to track the driver's location en-route to the pickup location and during the trip.
- **In-App Payment Gateway:** Integration of Razorpay/Stripe APIs to handle advance deposits and final settlements seamlessly within the app.
- **Automated Background Verification:** Implementing third-party APIs (like Digilocker integrations) to instantly verify Aadhaar and Driving License authenticity without Admin manual intervention.
- **In-App Chat Engine:** Enable secure, masked communication between driver and customer using WebSockets (Socket.io) to protect privacy.
- **Mobile Application Wrapper:** Utilizing React Native or Capacitor to package the progressive web app into downloadable Android and iOS applications.

19. BIBLIOGRAPHY

1. React 18 Official Documentation - <https://react.dev>
2. Fast2SMS API Reference & Integration Guide - <https://docs.fast2sms.com>
3. React Router DOM Routing Guidelines - <https://reactrouter.com>
4. Vite Next-Generation Build Tool - <https://vitejs.dev>
5. Lucide-React Icons Library - <https://lucide.dev>
6. Modern Glassmorphism CSS Techniques - <https://css-tricks.com>